United International University

Department of Computer Science and Engineering

Object Oriented Programming Laboratory

Topic: Classes, Objects, Constructors, & Static Members Duration: 2.5 hours

Lab Sheet: Classes, Objects & Constructors

Prepared by: Ashraful Islam Paran, Dept. of CSE, UIU. Follow instructions carefully and complete all assignments within the designated slot.

1. Objectives

By the end of this lab, students will be able to:

- Define custom templates using classes and instantiate them as runtime objects.
- Understand, design, and invoke default, parameterized, and copy constructors.
- Apply constructor overloading principles within structured definitions.
- Analyze and distinguish between object instances and global `static` members.
- Allocate, initialize, and traverse complex reference structures through arrays of objects.

2. Introduction

In Java, a **class** acts as a design blueprint, defining state fields and functional operations. An **object** represents a live allocation of that design container inside computer memory.

Key Components:

- **Constructor**: A structural initialization block invoked automatically during an object's instantiation.
- **Constructor Overloading**: Implementing multiple constructor variants varying by argument parameters.
- **Copy Constructor**: A specific architectural setup that instantiates an object by extracting values from a pre-existing sibling instance.
- **static** keyword: Binds field states or execution functions directly to the template blueprint level instead of independent object locations.

3. Example 1: Constructors & Constructor Overloading

ConstructorOverloadingAndCopyDemo.java

```java
class BookExample {
    String title;
    int pages;

    BookExample() {
        title = "No Title";
        pages = 0;
    }

    BookExample(String title) {
        this.title = title;
        pages = 0;
    }

    BookExample(String title, int pages) {
        this.title = title;
        this.pages = pages;
    }

    BookExample(BookExample other) {
        title = other.title;
        pages = other.pages;
    }

    void display() {
        System.out.println("Title: " + title + ", Pages: " + pages);
    }
}

public class ConstructorOverloadingAndCopyDemo {
    public static void main(String[] args) {
        BookExample b1 = new BookExample();
        BookExample b2 = new BookExample("Java Basics");
        BookExample b3 = new BookExample("OOP in Action", 250);
        BookExample b4 = new BookExample(b2);

        b1.display();
        b2.display();
        b3.display();
        b4.display();
    }
}
```

4. Example 2: Static Members

StaticMembersDemo.java

```
class StudentExample {
    String name;
    int id;

    static String dept = "CSE";
    static String uni = "UIU";
    static int totalStudents = 0;

    int perObjectCounter = 0;

    StudentExample() {
        totalStudents++;
        perObjectCounter++;
    }

    void displayName() {
        System.out.println("Name: " + name);
    }

    static void showStaticInfo() {
        System.out.println("Department: " + dept + ", University: " + uni);
    }
}

public class StaticMembersDemo {
    public static void main(String[] args) {
        System.out.println("Initial total students: " + StudentExample.
   totalStudents);

        StudentExample s1 = new StudentExample();
        s1.name = "Alice";

        StudentExample s2 = new StudentExample();
        s2.name = "Bob";

        System.out.println("Total students: " + StudentExample.totalStudents);
        System.out.println("s1 counter: " + s1.perObjectCounter);
        System.out.println("s2 counter: " + s2.perObjectCounter);

        StudentExample.showStaticInfo();
    }
}
```

5. Example 3: Array of Objects

PointArrayDemo.java

```java
class PointExample {
    int x;
    int y;

    PointExample(int x, int y) {
        this.x = x;
        this.y = y;
    }

    void display() {
        System.out.println("(" + x + ", " + y + ")");
    }
}

public class PointArrayDemo {
    public static void main(String[] args) {
        PointExample[] points = new PointExample[5];
        for (int i = 0; i < points.length; i++) {
            points[i] = new PointExample(i, i + 1);
        }

        System.out.println("Array of Points:");
        for (PointExample p : points) {
            p.display();
        }
    }
}
```

6. Example 4: Circle Class with Methods

CircleCalculationsDemo.java

```
class CircleExample {
    int radius;

    CircleExample(int radius) {
        this.radius = radius;
    }

    double area() {
        return Math.PI * radius * radius;
    }

    double circumference() {
        return 2 * Math.PI * radius;
    }
}

public class CircleCalculationsDemo {
    public static void main(String[] args) {
        CircleExample c1 = new CircleExample(7);
        System.out.printf("Radius: %d%n", c1.radius);
        System.out.printf("Area: %.2f%n", c1.area());
        System.out.printf("Circumference: %.2f%n", c1.circumference());
    }
}
```

7. Example 5: Object Composition (Has-A Relationship)

LibraryDemo.java

```
class BookExample {
    String title;
    int pages;

    // Default Constructor
    BookExample() {
    }

    // Parameterized Constructor
    BookExample(String title, int pages) {
        this.title = title;
        this.pages = pages;
    }

    void display() {
        System.out.println("Title: " + title);
        System.out.println("Pages: " + pages);
    }
```

```
}

class Student {
    String name;
    int id;
    double cgpa;

    Student() {
        name = "Unknown";
        id = -1;
        cgpa = 2.0;
    }

    Student(String name, int id, double cgpa) {
        this.name = name;
        this.id = id;
        this.cgpa = cgpa;
    }

    void display() {
        System.out.println("Name: " + name);
        System.out.println("ID: " + id);
        System.out.println("CGPA: " + cgpa);
    }
}

class Library {
    Student student;
    BookExample book;

    Library() {
    }

    Library(Student student, BookExample book) {
        this.student = student;
        this.book = book;
    }

    void display() {
        System.out.println("Student Information");
        student.display();

        System.out.println();

        System.out.println("Book Information");
        book.display();
    }
}

public class LibraryDemo {
    public static void main(String[] args) {
```

```
        Student s = new Student("Sakib", 12, 3.44);
        BookExample b = new BookExample("Java Programming", 366);

        Library library = new Library(s, b);

        library.display();
    }
}
```

Tasks & Solutions

Task 1: Book Class (15 min) Enhance the Book class with:

- A method to calculate reading time (pages / 20 pages per hour).
- Create 3 different books using various constructors.

Task 2: Student Management (20 min) Create a `Student` class with:

- Static variable for total students.
- Instance variables: name, id, cgpa.
- Constructor and static method to show university info.

Task 3: Array of Circles (20 min)

a. Create an array of 5 Circle objects with different radii.

b. Print area and circumference for each circle.

c. Calculate total area of all circles.

Task 4: Comprehensive Exercise (30 min) Design a `BankAccount` class with:

- Account number, balance (instance).
- Static interest rate.
- Constructors (default, parameterized, copy).
- Methods: deposit, withdraw, display.
- Create multiple accounts and demonstrate all features.

Keep Coding!

Understanding Classes and Objects is the foundation of Object-Oriented Programming in Java.